



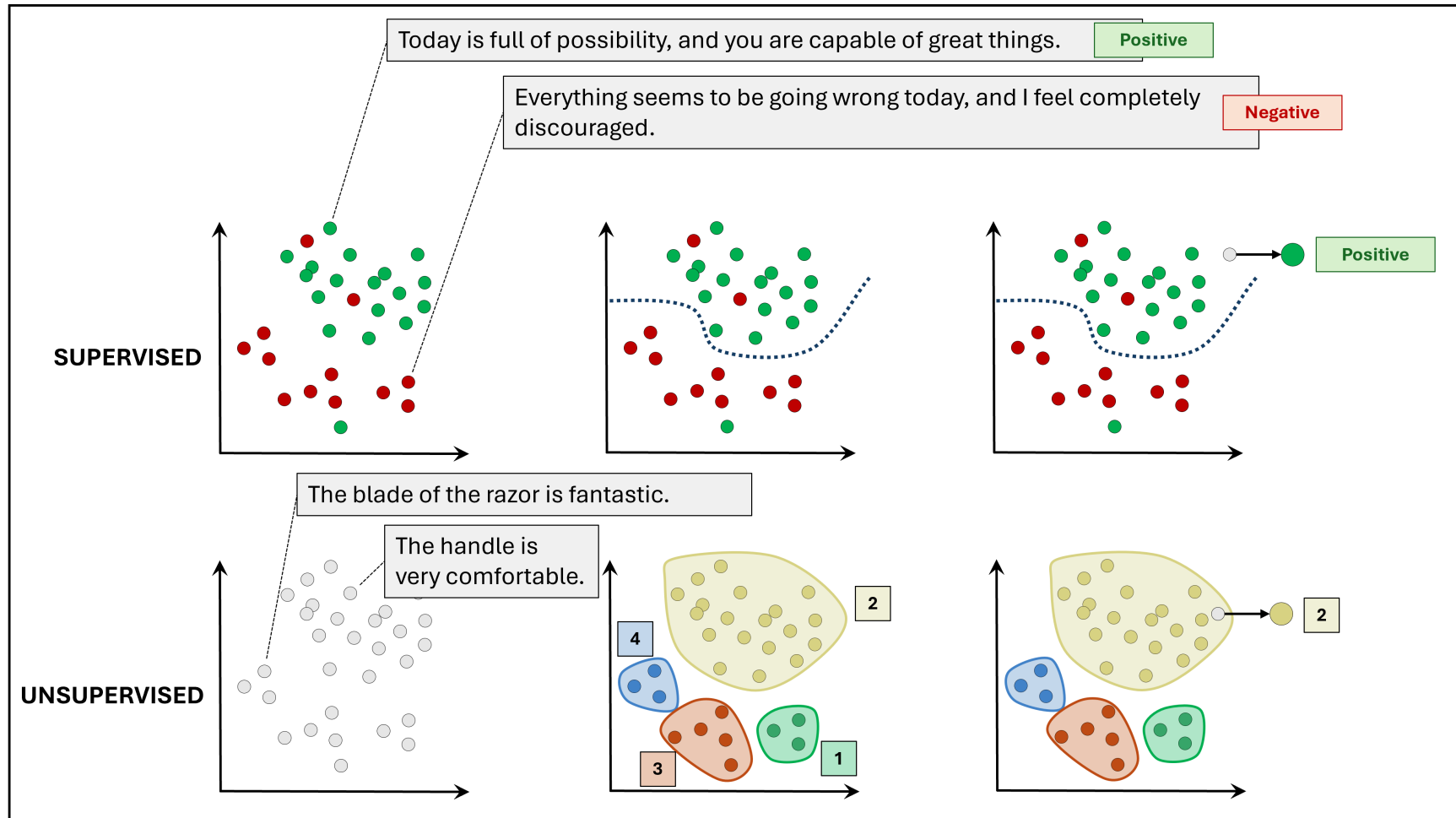
Text Modelling

Filippo Chiarello & Vito Giordano

Natural Language Processing Tasks

- **Supervised tasks:** A supervised NLP task is a task in which each text input is associated with a *ground truth*, that is, an expected output, reference label, or human annotation. The objective is to produce outputs consistent with the available ground truth for new inputs.
- **Unsupervised tasks:** An unsupervised NLP task is a task in which text data are available *without ground truth*, meaning that no expected output or reference annotation is provided for each instance. The objective is to identify patterns, structures, groups, or regularities in the text.

Natural Language Processing Tasks



Natural Language Processing Tasks

Supervised Tasks:

- Document Classification
- Sentence Classification
- Named Entity Recognition
- Relation Extraction
- Sentiment Analysis

Unsupervised Tasks:

- Topic Modelling
- Clustering
- Text Similarity

Dataset

First we have to select a dataset.

We selected a **dataset of tweets** from Kaggle that captures users' perceptions of ChatGPT.



Source:

[The ChatGPT Tweets Dataset](#)

For our analysis, we selected a sample of **1,000 tweets**.



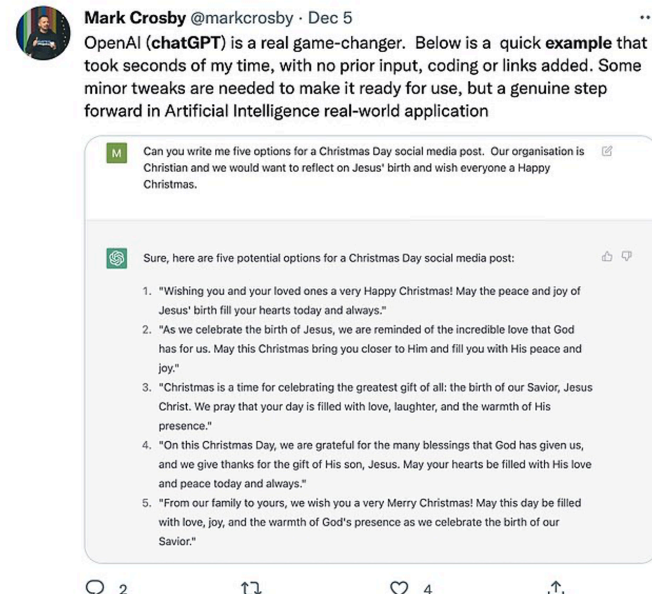
Dataset

This dataset contains the tweets which talks about how ChatGPT can be used for different tasks.



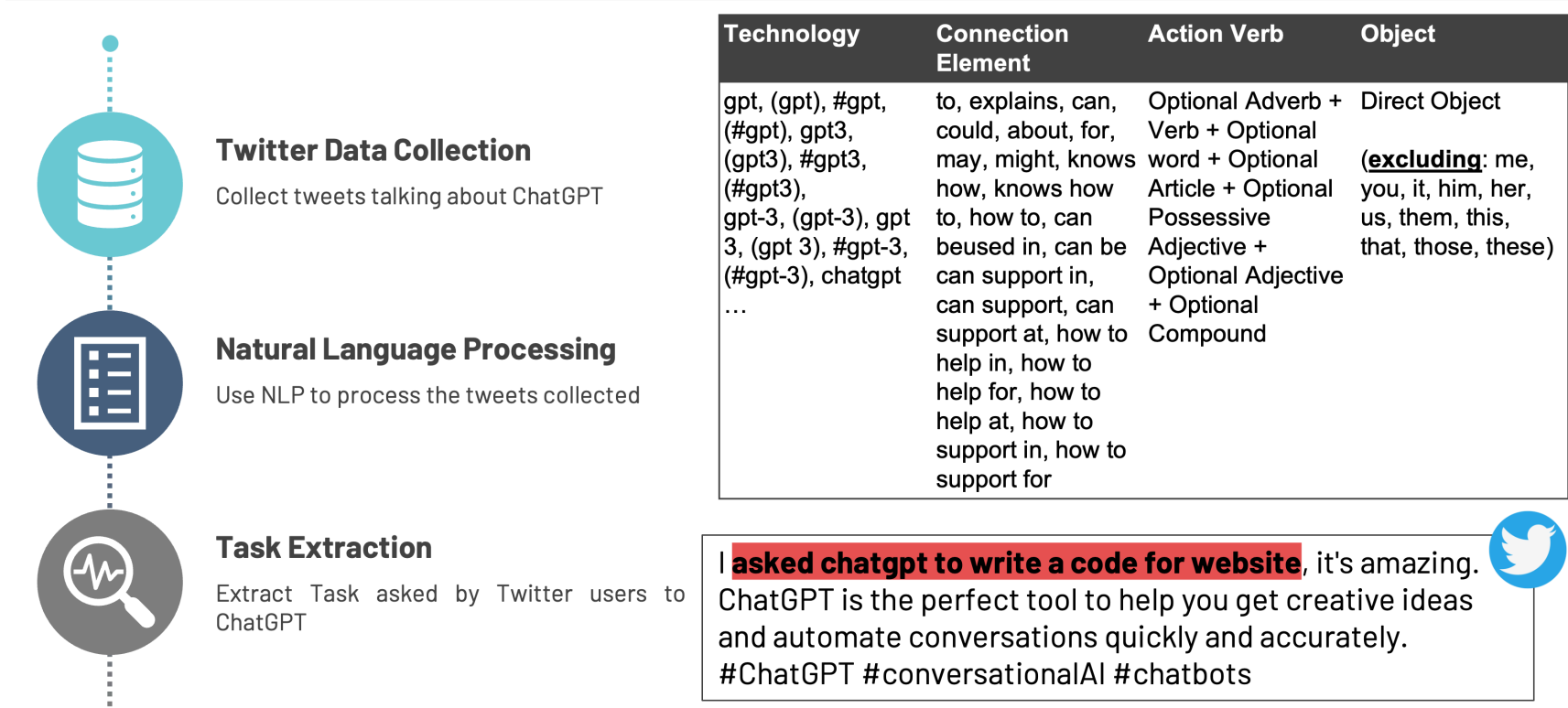
QUERY: keywords **chatgpt** or **gpt**

The collection through Twitter API brought to the generation of a dataset containing **3 mln** tweets.



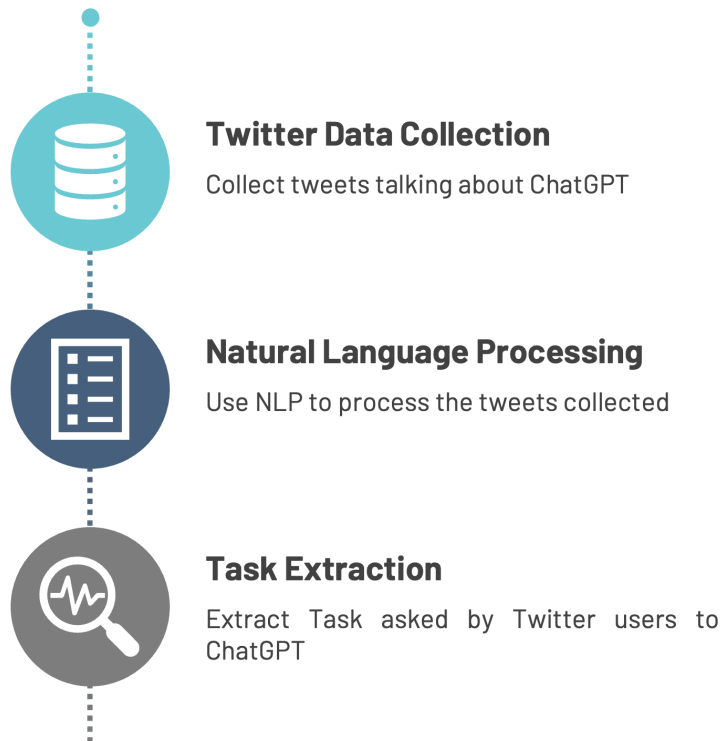
Dataset

We can see here the method applied for extracting the ChatGPT tasks.



Dataset

And also here some results of the tasks extraction.



Top-20 tasks for number of tweets.

Task	# of tweets	Task	# of tweets
Write a poem	887	Gives answers	269
Write code	775	Write a script	234
Write an essay	617	Write tool	225
Write a song	509	Passed a wharton mba exam	220
Answer questions	477	Affect web3 space	212
Write story	463	Do work	196
Write article	332	Generate content	184
Write a tweet	326	Mean for the future	174
Generate code	301	Gave the correct answer	161
Generate text	296	Write emails	159

9,558 tweets have at least one task.

5,554 different tasks are collected.

Dataset



We now import the dataset.

```
#{r}  
# Import packages  
library(tidyverse)  
  
# Import tweet  
tweet_data <- read_csv("data/sample_tweet.csv")
```

```
## # A tibble: 4 × 16  
##   tweet_id original_text      sentiment tasks users technologies  
##   <dbl> <chr>           <chr>    <chr> <chr> <chr>  
## 1  1.60e18 "[GPT-3] This post... neutral   ensu... <NA> artificial ...  
## 2  1.60e18 "Today is the pre-... neutral   unde... <NA> database  
## 3  1.60e18 "@DeclanMcCreary T... negative inve... acad... model  
## 4  1.60e18 "Screenshot of a p... neutral   wrot... <NA> model  
## # i 10 more variables: organizations <chr>,  
## #   job_competencies <chr>, job_profiles <chr>, nouns <chr>,  
## #   verbs <chr>, adjectives <chr>, subjects <chr>,  
## #   objects <chr>, predicates <chr>, topic <chr>
```

Dataset



Let's select only useful information.

```
#{r}
# Select only useful information
tweet_data <- tibble(tweet_data) |>
  select(tweet_id, original_text, tasks)
```

```
## # A tibble: 6 × 3
##   tweet_id original_text          tasks
##   <dbl> <chr>                  <chr>
## 1  1.60e18 "[GPT-3] This post discusses the need for an au... ensu...
## 2  1.60e18 "Today is the pre-fine-tuning day! \r\n\r\nI'm ... unde...
## 3  1.60e18 "@DeclanMccreary They don't program it to inven... inve...
## 4  1.60e18 "Screenshot of a project I'm working on using t... wrot...
## 5  1.60e18 "@Vkumzy @MsRabbitKick @rippleitinNZ It's OpenA... writ...
## 6  1.60e18 "@Andrew___Baker When I tried it, GPT-3 nailed ... nail...
```

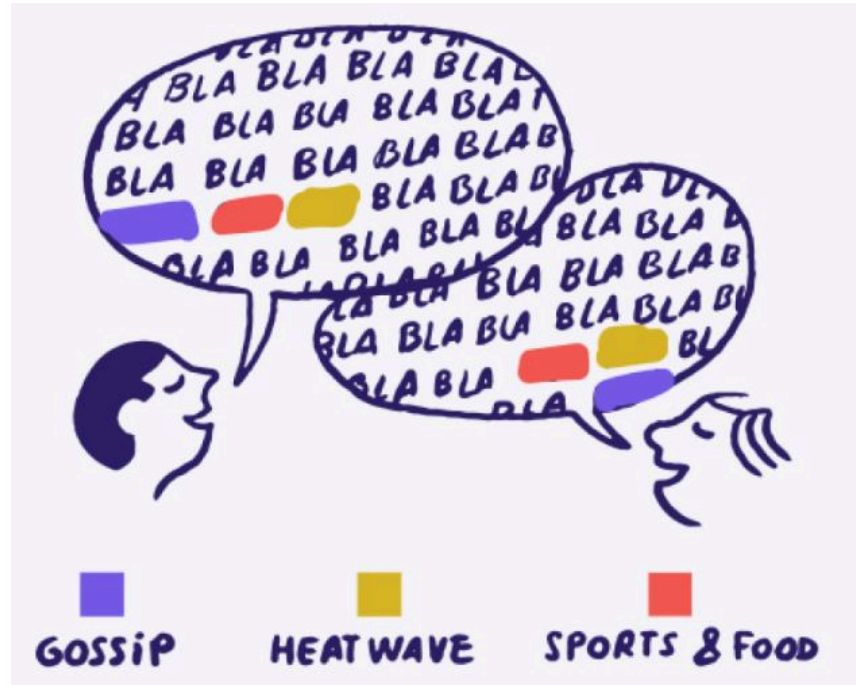


Topic Modelling

Topic Modelling

Topic Modelling is a Unsupervised task which aims to automatically extract thematic information from text.

What is the main topic of a document?

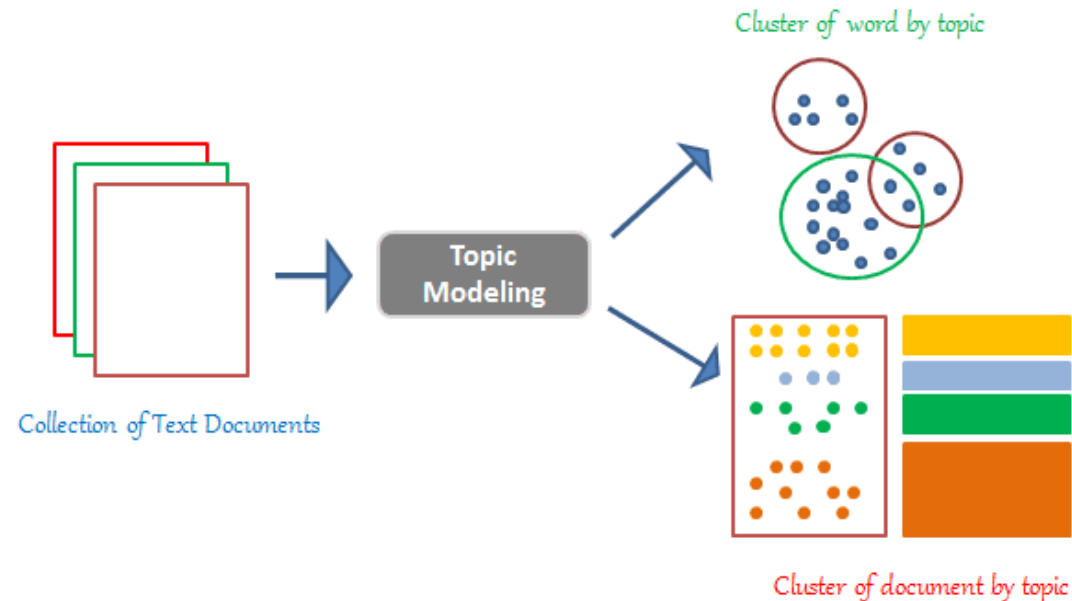


Topic Modelling - Applications

- **Document Clustering:** Organizing large sets of documents (e.g., academic papers, news articles) into clusters based on topics.
- **Content Recommendation:** Recommending content to users based on their interests.
- **Customer Feedback Analysis:** Analyzing customer feedback to identify common themes and issues.

Topic Modelling

It usually employs **unsupervised** techniques (the algorithms do not need prior labeling and annotation of documents – the topics emerge from the analysis of original texts).

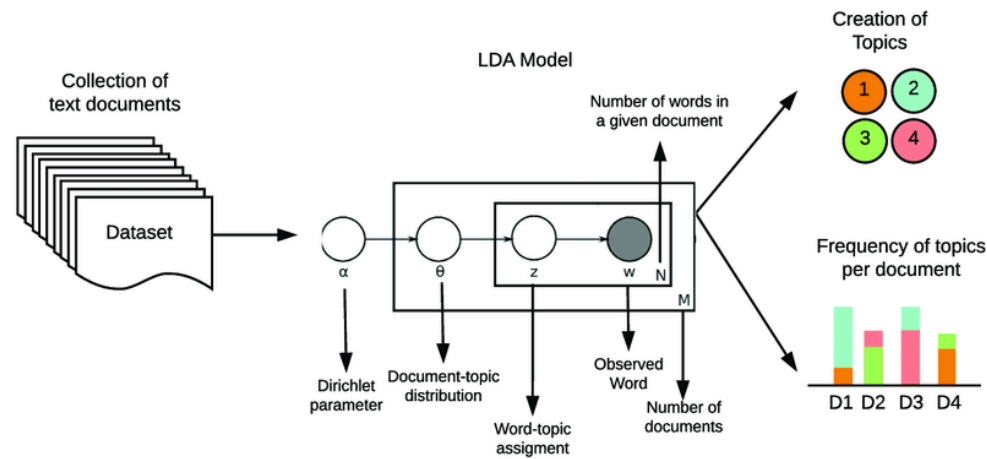


The most used topic modeling method is **Latent Dirichlet Allocation (LDA)**.

Latent Dirichlet allocation (LDA)

Latent Dirichlet allocation (**LDA**) is one of the most common algorithms for topic modeling. Without diving into the math behind the model, we can understand it as being guided by two principles:

- **Every document is a mixture of topics.** We imagine that each document may contain words from several topics in particular proportions. This portion is the *per-document-per-topic probabilities*, called γ (“gamma”).
- **Every topic is a mixture of words.** Importantly, words can be shared between topics; a word like “budget” might appear in both equally. The likelihood that a word belongs to a given topic is the *per-topic-per-word probabilities*, called β (“beta”).



Latent Dirichlet allocation (LDA)

LDA is a **bag-of-words** model, meaning that the sequence of words in documents is not important for the model. Selecting the number of topics is one of the choices researchers have to make. However, no exact guidelines for this choice exist and the effect of the number of topics on the robustness of LDA results is not well understood.

Input of LDA: Document-Term Matrix

One of the most common structures that text mining packages work with is the document-term matrix (or DTM). This is a matrix where:

- **each row** represents one **document** (such as a book or article),
- **each column** represents one **term**, and
- **each value** (typically) contains the **number of appearances** of that term in that document.

	text	mining	is	to	find	useful	information	from	text	mined	dark	came
D1	1	1	1	1	1	1	1	1	1	0	0	0
D2	0	0	1	0	0	1	1	1	1	1	0	0
D3	0	0	0	0	0	0	0	0	0	0	1	1

- `tidy()` **turns a document-term matrix into a tidy data frame**. This verb comes from the broom package (Robinson 2017), which provides similar tidying functions for many statistical models and objects.
- `cast()` **turns a tidy one-term-per-row data frame into a matrix**. tidytext provides three variations of this verb, each converting to a different type of matrix: `cast_sparse()` (converting to a sparse matrix from the Matrix package), `cast_dtm()` (converting to a DocumentTermMatrix object from tm), and `cast_dfm()` (converting to a dfm object from quanteda).

Topic Modeling in Practice

We employ the AssociatedPress dataset as an example of a DocumentTermMatrix. This is a collection of 2246 news articles from an American news agency, mostly published around 1988.

```
library(topicmodels)
library(tidyverse)
library(tidytext)

data("AssociatedPress")
AssociatedPress
```

```
## <<DocumentTermMatrix (documents: 2246, terms: 10473)>>
## Non-/sparse entries: 302031/23220327
## Sparsity           : 99%
## Maximal term length: 18
## Weighting          : term frequency (tf)
```

Topic Modeling in Practice

We can use the **LDA()** function from the **topicmodels** package, setting $k = 2$, to create a two-topic LDA model. The number of topics (k) is decided by us (2 in this case).

```
ap_lda <- LDA(AssociatedPress, k = 2, control = list(seed = 1234))  
ap_lda
```

```
## A LDA_VEM topic model with 2 topics.
```

Fitting the model was the “easy part”: the rest of the analysis will involve exploring and interpreting the model using tidying functions from the tidytext package.

Word-topic probabilities

The tidytext package provides the method **tidy()** for extracting the per-topic-per-word probabilities, called **β** (“beta”), from the model.

```
# Extract Beta: per-topic-per-word probabilities
ap_topics <- tidy(ap_lda, matrix = "beta")
ap_topics
```

```
## # A tibble: 20,946 × 3
##   topic term      beta
##   <int> <chr>    <dbl>
## 1     1 aaron  1.69e-12
## 2     2 aaron  3.90e- 5
## 3     1 abandon 2.65e- 5
## 4     2 abandon 3.99e- 5
## 5     1 abandoned 1.39e- 4
## 6     2 abandoned 5.88e- 5
## # i 20,940 more rows
```

Notice that this has turned the model into a **one-topic-per-term-per-row** format. For each combination, the model computes the probability of that term being generated from that topic.

Word-topic probabilities

We could use dplyr's `slice_max()` to find the 10 terms that are most common within each topic.

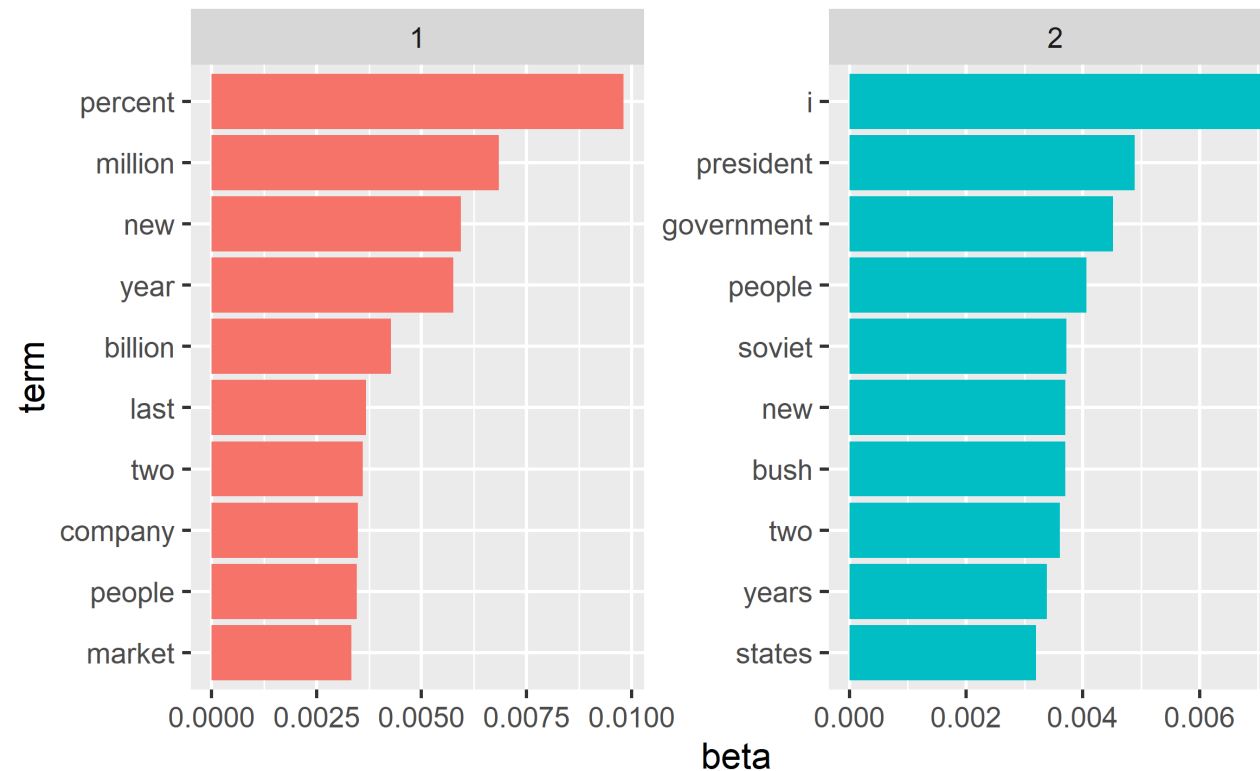
```
# Select the most occurred words in each topic
ap_top_terms <- ap_topics |>
  group_by(topic) |>
  slice_max(beta, n = 10) |>
  ungroup() |>
  arrange(topic, -beta)
```

As a tidy data frame, this lends itself well to a ggplot2 visualization

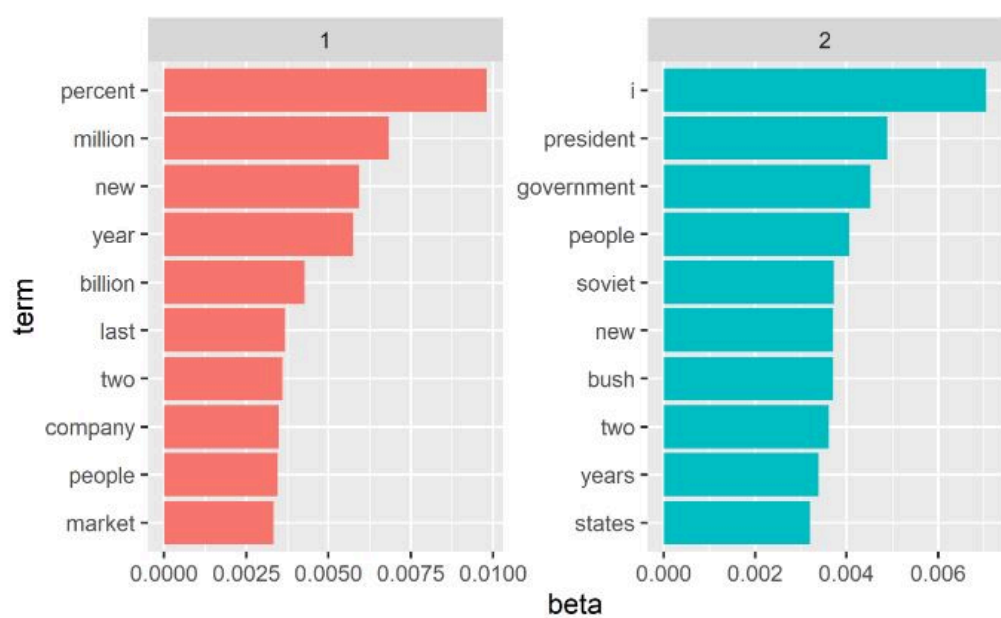
Word-topic probabilities



```
# Plot the results
ap_top_terms |>
  mutate(term = reorder_within(term, beta, topic)) |>
  ggplot(aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ topic, scales = "free") +
  scale_y_reordered()
```



Word-topic probabilities



This visualization lets us understand the two topics. The most common words in topic 1 include “percent”, “million”, “billion”, and “company”, which suggests it may represent **business or financial news**. Those most common in topic 2 include “president”, “government”, and “soviet”, suggesting that this topic represents **political news**. One important observation about the words in each topic is that some words, such as “new” and “people”, are common within both topics. This is an advantage of topic modeling as opposed to “hard clustering” methods: topics used in natural language could have some overlap in terms of words.

Document-per-topic probability

```
# Extract Gamma: per-document-per-topic probabilities
gamma_topics <- tidy(ap_lda, matrix = "gamma")
```

```
gamma_topics |>
  head(10)
```

```
## # A tibble: 10 × 3
##   document topic    gamma
##   <int> <int>    <dbl>
## 1     1     1  0.248
## 2     2     1  0.362
## 3     3     1  0.527
## 4     4     1  0.357
## 5     5     1  0.181
## 6     6     1  0.000588
## # i 4 more rows
```


LDA on our dataset

Let's employ pre-processing operations and LDA to perform Topic Modelling on Twitter.

```
library(udpipe)

# Load Model
ud_model <- udpipe_load_model("data/english-ewt-ud-2.5-191206.udpipe")

# Tokenize and PoS tagging
twitter_POS <- udpipe_annotate(ud_model, x = tweet_data$original_text, doc_id = tweet_data$tweet_id,
  as_tibble())

# Calculate the occurrences of each word for each tweets
twitter_words <- twitter_POS |>
  filter(upos == "NOUN") |>
  count(doc_id, lemma, sort = TRUE) # Keep only NOUN
```

LDA on our dataset



Let's remove stop words and then create the Document Term Matrix with **cast_dtm()**

```
# remove stopwords
twitter_words <- twitter_words |>
  filter(!(lemma %in% stop_words$word))

# Document Term-Matrix
twitter_dtm <- twitter_words |>
  cast_dtm(doc_id, lemma, n)
```

LDA on our dataset



Let's fit the LDA model, setting the number of topics to 6.

```
twitter_lda <- LDA(twitter_dtm, k = 6, control = list(seed = 1234))

# Extract Beta: per-topic-per-word probabilities
twitter_topics <- tidy(twitter_lda, matrix = "beta")

twitter_top_terms <- twitter_topics |>
  group_by(topic) |>
  slice_max(beta, n = 10) |>
  ungroup() |>
  arrange(topic, -beta)
```

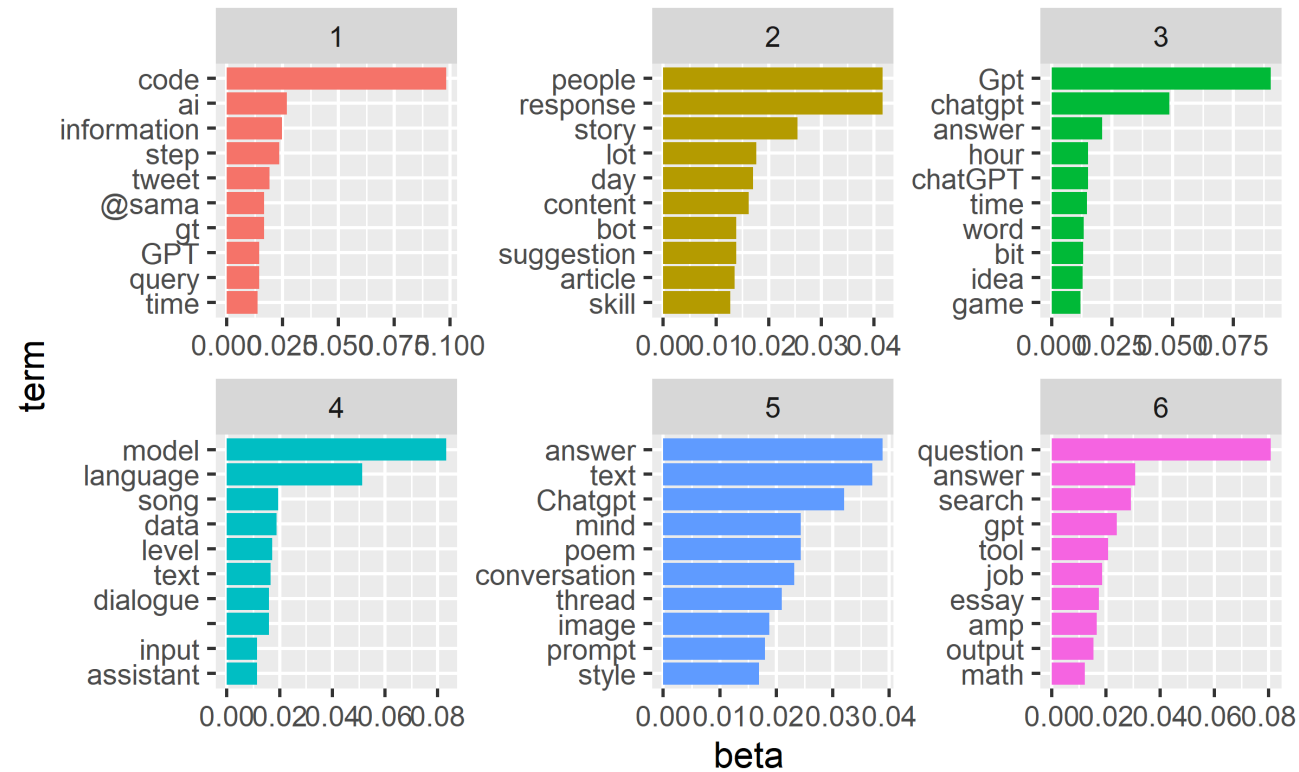
Word-topic probabilities



The most important words in each topic.

```
# Plot the results
twitter_top_terms |>
  mutate(term = reorder_within(term, beta, topic)) |>
  ggplot(aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ topic, scales = "free") +
  scale_y_reordered()
```

Word-topic probabilities



Document-per-topic probability

```
# Extract Gamma: per-document-per-topic probabilities
twitter_topics <- tidy(twitter_lda, matrix = "gamma")

# Example
twitter_topics |>
  left_join(tweet_data |> rename(document = tweet_id) |> mutate(document = as.character(document)))
head(5)
```

A tibble: 5 × 5

##	document	topic	gamma	original_text	tasks
##	<chr>	<int>	<dbl>	<chr>	<chr>
## 1	1598283202769977344	1	0.0136	"@OpenAI launch ChatGP...	here...
## 2	1598331165106135040	1	0.0151	"AI / ML news of the W...	codi...
## 3	1598059818845368320	1	0.0124	"8. Stable Diffusion v...	requ...
## 4	1598079536372936704	1	0.0152	"AI can design games a...	desi...
## 5	1598182712783040512	1	0.00901	"have been using GPT-3...	adap...

Feature selection

We can apply a series of words unification and removal steps, to select the relevant words for representing each abstract (i.e., features). We thus removed:

- Sparse terms occurring in less than 0.1% of all documents;
- Common terms occurring in more than 10% of all documents;
- List of domain-related stop-words (e.g., abstract, paper, research);
- Short words, composed of less than three characters;
- Every part of speech that does not belong to nouns and adjectives.

Topic Modelling Evaluation



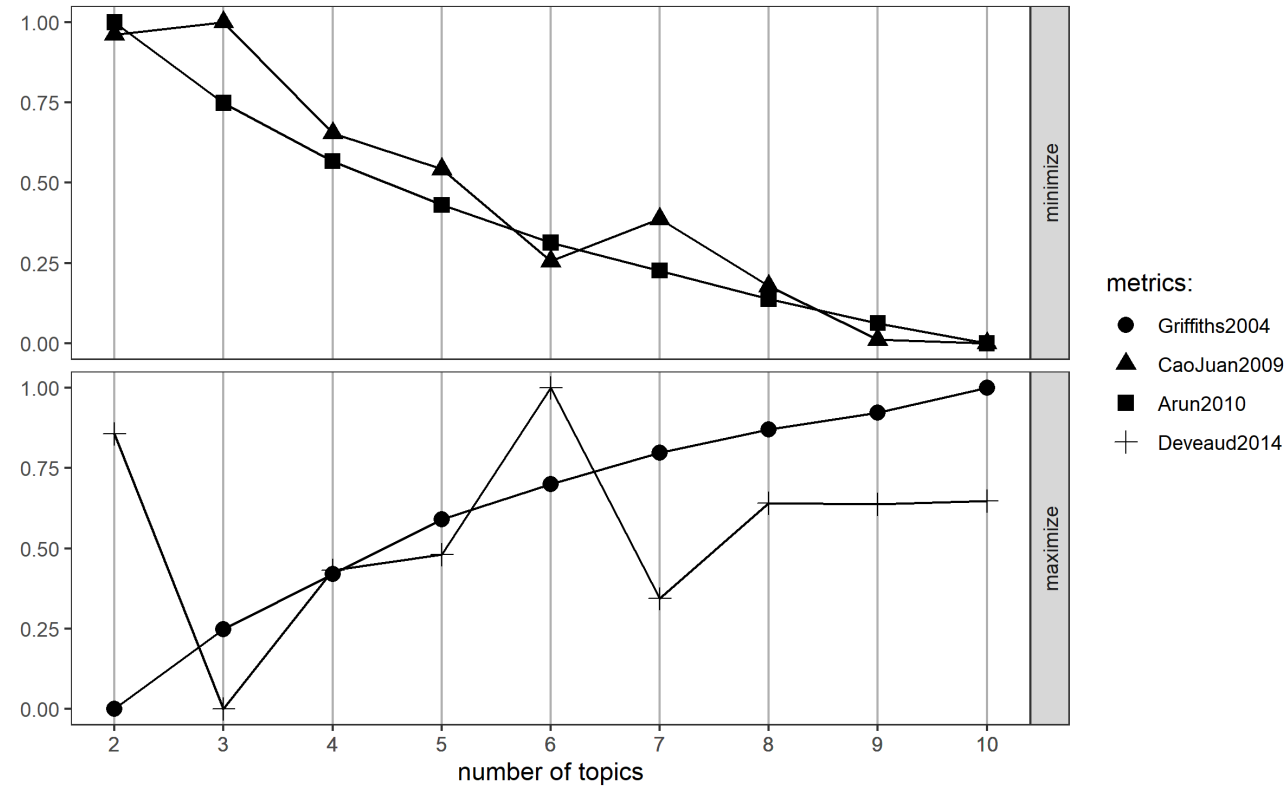
To evaluate the model we used the following packages

```
library(topicmodels)
library(lдатuning)
```

```
# Set the number max of topics to evaluate in "to" (in this case equal to 10)
result <- FindTopicsNumber(
  twitter_dtm,
  topics = seq(from = 2, to = 10, by = 1),
  metrics = c("Griffiths2004", "CaoJuan2009", "Arun2010", "Deveaud2014"),
  method = "Gibbs",
  control = list(seed = 77),
  mc.cores = 2L,
  verbose = TRUE
)
```


Topic Modelling Evaluation

```
FindTopicsNumber_plot(result)
```



Latent Dirichlet allocation (LDA) - Limitations

Bag-of-words models have one significant limitation. **Unigrams** are sometimes difficult to interpret. Certain words often appear together to form phrases (e.g. 'competitive advantage', 'human resource management'). This limitation can be dealt with through two distinct approaches:

- LDA extension model that can incorporate n-grams. Use **preprocessing** to calculate n-grams as terms and include them in Document-Term matrix which is later processed with normal LDA or other topic modeling algorithm.

BERTopic

An other algorithm of Topic Modelling is [BERTopic](#). BERTopic is a topic modelling technique that leverages **transformers** to create dense clusters allowing for easily interpretable topics whilst keeping important words in the topic descriptions.

BERTopic - Set the environment

For running BERTopic we have to use Python packages **transformers** and **torch**. To do it, you need a Python environment that can be called from R through the package **reticulate**.

As Python environment, we suggest to install [Anaconda3](#).



BERTopic - Set the enviroment



Once Anaconda is installed, you should also install the libraries **transformers** and **torch** through the Anaconda Prompt:

```
pip install pytorch  
pip install transformers
```

Then, install reticulate in RStudio.

```
#{r}  
#install.packages("reticulate")
```

BERTopic - Set the environment

Set the python environment.

```
{r}
library(reticulate)

# Use the desired Python environment (adjust the path to your specific environment)
use_python("C:/Users/Vito Giordano/anaconda3/python.exe", required = TRUE)
py_config()
```

```
## python:      C:/Users/Vito Giordano/anaconda3/python.exe
## libpython:   C:/Users/Vito Giordano/anaconda3/python311.dll
## pythonhome:  C:/Users/Vito Giordano/anaconda3
## version:     3.11.4 | packaged by Anaconda, Inc. | (main, Jul  5 2023, 13:38:37) [MSC v.1916 64 bit
## Architecture: 64bit
## numpy:       C:/Users/Vito Giordano/anaconda3/Lib/site-packages/numpy
## numpy_version: 1.24.3
##
## NOTE: Python version was forced by use_python() function
```

BERTopic - Set the enviroment



Set the python environment.

```
{r}  
# Import transformers library  
transformers <- import("transformers")  
torch <- import("torch")
```

Transformers is a library developed by Hugging Face that provides pre-trained models and tools for NLP.

PyTorch is a machine learning library developed by Facebook that provides tools for deep learning and tensor computation.

BERTopic

To prepare the data for Topic modeling, we first select only the tasks. In this case we have to merge all the tasks in a unique text for finding the most frequent topics.

```
#{r}  
# Select only the tweet text  
topic_data <- tweet_data |>  
  select(tasks) |>  
  distinct() |>  
  pull(tasks)
```

```
## [1] "ensure safety"  
## [2] "understand a simple mysql database"  
## [3] "invent citations, it can construct language"  
## [4] "wrote a paragraph"  
## [5] "write an excuse letter"  
## [6] "nailed a number"
```


BERTopic



We use the **BERTopic** library in Python via reticulate. BERTopic generates dense embeddings and applies clustering to detect meaningful topics.

```
{python}
from bertopic import BERTopic

# Convert R dataset in python
topic_data = r.topic_data

# Create and fit the BERTopic model
topic_model = BERTopic(language="english")

# Find the topics
topics = topic_model.fit_transform(topic_data)
```

BERTopic



We can now inspect the results from the BERTopic model.

```
{python}
# Show top topics with frequency
topic_info = topic_model.get_topic_info()
topic_info.head()
```

```
## # A tibble: 6 × 5
##   Topic Count Name      Representation Representative_Docs
##   <dbl> <dbl> <chr>      <chr>      <chr>
## 1     -1   273 -1_the_build_to... ['the', 'buil... ['using openai cha...
## 2      0   229 0_answer_gives_... ['answer', 'g... ['actually answers...
## 3      1    81 1_write_text_st... ['write', 'te... ['write a', 'write...
## 4      2    59 2_explaining_it... ['explaining'... ['explaining the t...
## 5      3    35 3_code_coding_r... ['code', 'cod... ['generated code',...
## 6      4    31 4_chatgpt_profi... ['chatgpt', '... ['try chatgpt', 'r...
```

BERTopic



We can analyse the most frequent topic found by BERTopic.

But first, we have to remove the outlier topics.

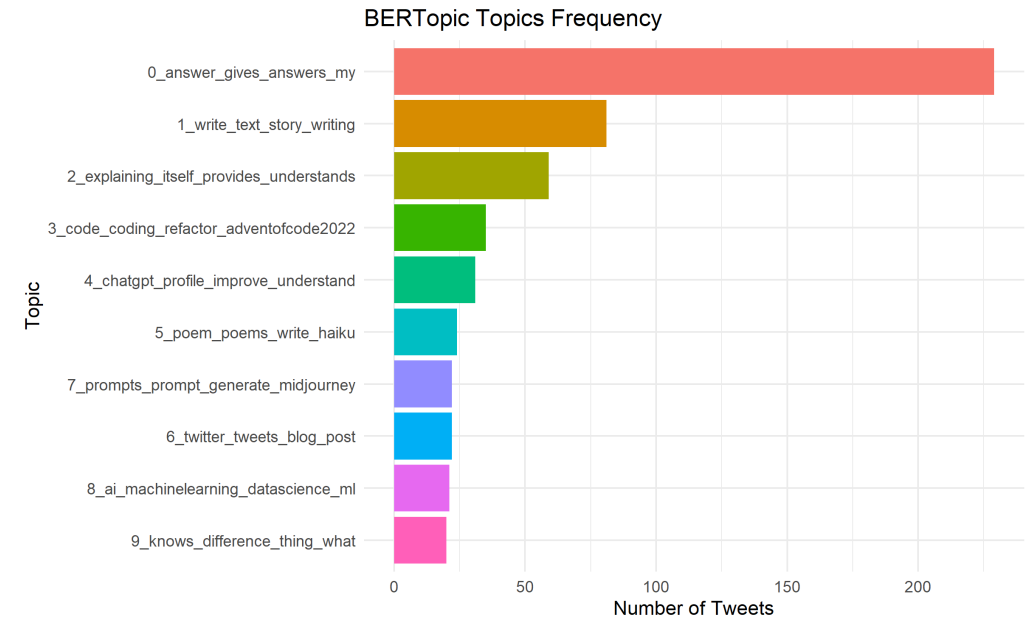
```
{r}
# Filter out the outlier topic (-1), sort by count, and select the top 10 topics
topic_info_filtered <- topic_info |>
  filter(Topic != -1) |>
  arrange(desc(Count)) |>
  slice_head(n = 10)
```

BERTopic



Now we can analyse the most frequent topic found by BERTopic.

```
{r}  
library(ggplot2)  
  
# Plot most frequent Topic  
ggplot(topic_info_filtered,  
  aes(x = Count,  
      y = reorder(Name, Count),  
      fill = Name)) +  
  geom_bar(stat = "identity") +  
  labs(title = "BERTopic Topics Frequency",  
       x = "Number of Tweets",  
       y = "Topic") +  
  theme_minimal() +  
  theme(legend.position = "none")
```



BERTopic



We can also analyse the most frequent terms for the topics.

```
{python}  
topic_model.visualize_barchart(top_n_topics=8, n_words=8)
```

